

Giving teams their own folders using Regex

Instead of manually giving permissions to every single job, we use *Item Roles* and *Regular Expressions (Regex)* to automate access.

- In the same *Manage Roles* screen, scroll down to *Item Roles*.
- Create a role named *developer-alpha*.
- In the *Pattern* field, enter *Alpha-.**

The Regex secret: The *.** means ‘match anything’. By using *Alpha-.**, Jenkins automatically grants access to any job that starts with the prefix ‘Alpha-’.

- Check *Job/Read*, *Job/Build*, and *Job/Workspace*. This allows the *dev-*

user to see their project and run it, but they cannot change the job configuration.

- Repeat this for the *ops-team* using the pattern *.** (which matches every job on the server) and grant them all checkboxes under *Job*, *Run*, and *View*.

The example of item roles for the operations team is shown in Figure 2.

Keeping your secrets and passwords safe

A hardened Jenkins environment requires protecting sensitive data, such as API keys and GitHub tokens, by avoiding ‘hardcoding’

them directly into scripts. To achieve this, secrets should be stored using the *Credentials Provider* found under *Manage Jenkins > Credentials*. Jenkins stores these secrets in an encrypted format. Furthermore, with proper *Access Control* and *Item Roles* configured, a developer can utilise a credential in their code via a variable, but they are specifically prevented from viewing or editing the secret’s actual value within the Jenkins user interface, enhancing security.

Testing the setup: Does it actually work?

Once the roles are assigned, it is time to verify the ‘Least Privilege’ principle



Figure 2: Ops team roles and Regex



Figure 3: Testing the *ops-team* user